



Tecnológico de Monterrey

Instituto Tecnológico y de Estudios Superiores de Monterrey
Campus Puebla

TE3002B: Implementación de robótica inteligente (Gpo 501)

Dr. Alfredo García Suárez

Actividad 8.2 (SLAM de Lidar)

Marcos Allen Martínez Cortés

A01737939

26 de mayo de 2026

Implementar el código requerido para generar el seguimiento de los siguientes waypoints de forma aleatoria, ajustando los parámetros: `sampleTime`, `tVec`, `initPose`, `lidar.scanAngles`, `lidar.maxRange`, `waypoints`, `controller.LookaheadDistance`, `controller.DesiredLinearVelocity` y `controller.MaxAngularVelocity`. Evadiendo los obstáculos del mapa de navegación “`complexMap`” (Para evadir los obstáculos es posible agregar way-points intermedios en el recorrido).

Pasos para poder obtener el mejor desempeño en el seguimiento de los waypoints

El controlador Pure Pursuit solo puede ver el siguiente punto en la lista, por lo que, si los waypoints principales están muy separados o cambian de dirección bruscamente, el robot intentará acortar el camino siguiendo líneas rectas, pero atravesando las paredes y chocando los obstáculos.

Para poder obtener el mejor desempeño en el seguimiento de los waypoints, evadiendo los obstáculos del mapa de navegación “`complexMap`” debemos de seguir los siguientes pasos:

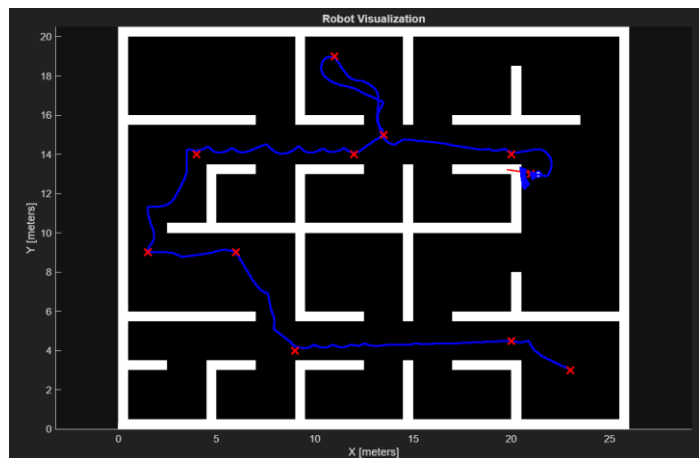
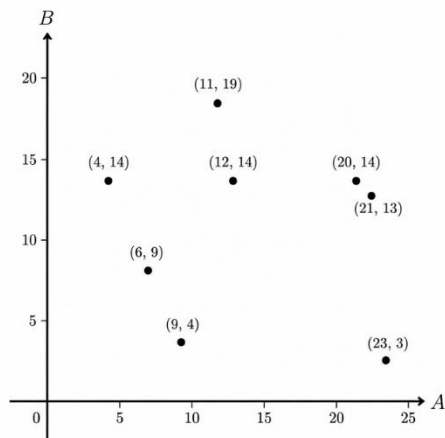
1. Planificar la ruta (orden de los waypoints), asegurando que se encuentren cercanos entre sí y evitando los obstáculos del mapa (en este caso, el `complexMap`).
2. Establecer la pose (posición y orientación) del robot con base en el primer waypoint de la ruta planificada.
3. Sintonizar el Pure Pursuit Controller, modificando los siguientes parámetros:
 - `LookaheadDistance`: es el parámetro más crítico, ya que, si es muy bajo, el robot intentará seguir la línea agresivamente provocando oscilaciones o giros sobre su propio eje. Por otra parte, si es muy alta, el robot recortará las esquinas y no pasará con precisión por los waypoints. Un valor de 0.5 podría ser un buen valor inicial, el cual se debería ir modificando.
 - `DesiredLinearVelocity` y `MaxAngularVelocity`: la velocidad lineal debe mantenerse moderada, ya que, si el robot va muy rápido, el robot no tendrá tiempo de reaccionar ante curvas cerradas u obstáculos. Por otra parte, es importante mantener una combinación de una velocidad lineal baja y una velocidad angular máxima alta, como 20 que venía en el código de ejemplo, que es lo que permite que el robot pueda reaccionar ante curvas cerradas y no colisionar.
4. Sintonizar el Lidar, modificando los siguientes parámetros:
 - `maxRange`: es el rango máximo que ve el Lidar, esto le permite al robot ver a distancia y empezar a corregir la trayectoria mucho antes de llegar al obstáculo, aunque un valor alto hará que, si un waypoint está cerca de un obstáculo, el robot no pase por el ya que estará reaccionando con el Lidar para evitar colisionar.

- **scanAngles:** es el campo de visión y la resolución del Lidar en la simulación; a través de dos ángulos determina la cobertura de su visión, el cual en el código está para una cobertura 360°, y la resolución, el tercer campo de 200, indica la cantidad de puntos en los que se divide ese rango.

Con este, se puede establecer el alcance angular y junto con el maxRange, que sería la magnitud, se puede configurar qué “ve” el Lidar.

5. Agregar waypoints intermedios según el comportamiento; esto, para hacer que si el robot sigue chocando en una esquina o sigue atravesando las paredes, un punto intermedio le permite cambiar su rumbo y tener una trayectoria más fluida. De este modo, añadir puntos que reduzcan la distancia entre puntos o el cambio de dirección brusco, le permite al robot tener un comportamiento adecuado; sobre todo al poder cuando el robot tiene que ser guiado en esquinas cerradas, para suavizar curvas o cuando esta rodeado de obstáculos fijos grandes; estos escenarios son donde se suelen utilizar waypoints intermedios.
6. Modificar el tiempo de simulación para que el robot pueda completar el seguimiento de los waypoints.

Primer conjunto de waypoints



Para este conjunto de waypoints en el complex Map, se utilizaron los waypoints asignados para construir una ruta. Y en la práctica se vio que se necesitarían de waypoints intermedios, porque el robot tenía puntos ciegos, por las paredes de obstáculos, o también, en un caso, para poder entrar y salir de un área casi completamente rodeada.

De este modo se utilizaron tres puntos intermedios:

- (20, 4.5)
- (1.5, 9)
- (13.5, 15), el cual se utilizó dos veces, para entrar y salir del área rodeada.

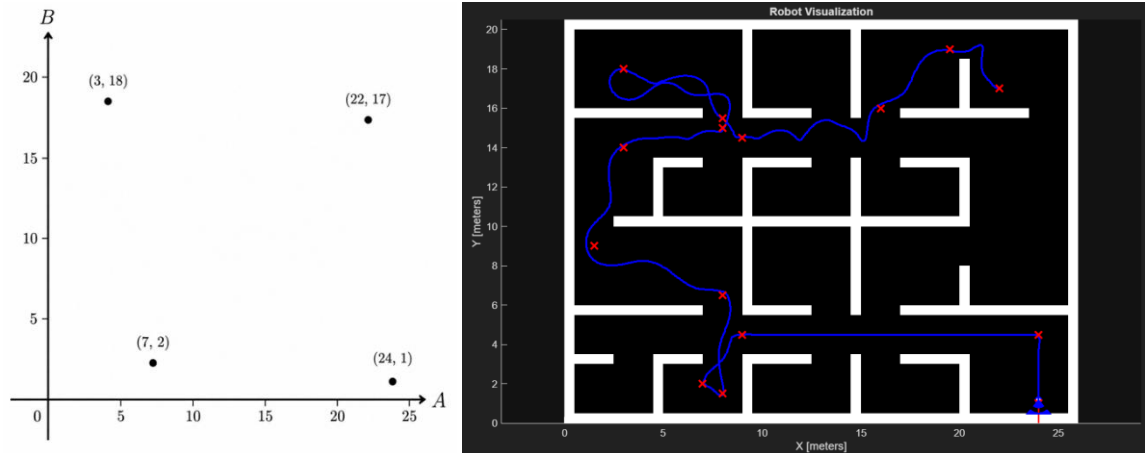
Posteriormente se sintonizaron los parámetros del controlador, asignando:

- LookaheadDistance de **0.35**, para que el robot pase adecuadamente por los waypoints.
- DesiredLinearVelocity de **0.5** y MaxAngularVelocity de **20**, que en combinación permitieron que el robot pudiera avanzar y durante el trayecto, reaccionar a tiempo a giros bruscos por el cambio de dirección al seguir a los waypoints o para evitar colisionar con los obstáculos del mapa.

Se ajustó el maxRange del Lidar a **1.25**, que en el código de ejemplo venía de 1; este aumento, permitió que el robot pudiera ver antes los obstáculos y tener tiempo para reaccionar.

Finalmente, se cambió el tiempo de simulación a **130 segundos**, para que el robot pudiera completar la ruta asignada con todos los waypoints principales (y los intermedios añadidos).

Segundo conjunto de waypoints



Similar al conjunto anterior; se utilizaron los waypoints asignados para construir una ruta, añadiendo waypoints intermedios cuando fue necesario. En este caso, como los puntos estaban muy separados entre sí y por lo mismo de que eran pocos, había varios obstáculos del mapa entre punto y punto, por lo que fue necesario añadir waypoints intermedios que guiaran con mayor claridad al robot.

De este modo se utilizaron los siguientes waypoints intermedios:

- (19.5, 19)
- (16, 16)
- (9, 14.5)
- (8, 15.5) y (8, 15), para entrar y salir del área rodeada.
- (3, 14)
- (1.5, 9)
- (8, 6.5)

- (8, 1.5)
- (9, 4.5)
- (24, 4.5)

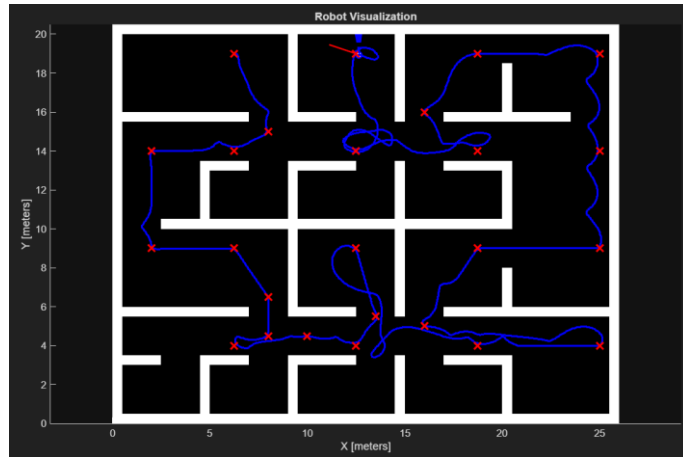
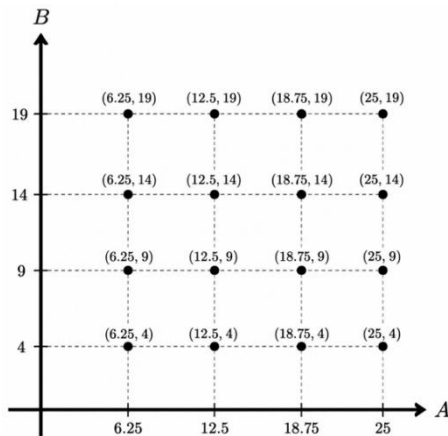
Posteriormente se sintonizaron los parámetros del controlador, asignando:

- LookaheadDistance de **0.35**, para que el robot pase adecuadamente por los waypoints, mismo valor utilizado anteriormente.
- DesiredLinearVelocity de **0.75** y MaxAngularVelocity de **20**, que en combinación permitieron que el robot pudiera avanzar con mayor rapidez y a la vez reaccionar a giros bruscos y evitar colisionar con los obstáculos del mapa.

Se utilizó el maxRange del Lidar de **1**, que venía en el código de ejemplo.

Finalmente, se cambió el tiempo de simulación a **111 segundos**.

Tercer conjunto de waypoints



Similar a los conjuntos anteriores; se utilizaron los waypoints asignados para construir una ruta, añadiendo waypoints intermedios cuando fue necesario. En este caso, dado que eran muchos waypoints, se utilizaron más que nada para los puntos ciegos y cuando el robot tenía que entrar o salir de áreas rodeadas.

De este modo se utilizaron los siguientes waypoints intermedios:

- (8, 15)
- (2, 14)
- (2, 9)
- (8, 6.5)
- (8, 4.5)
- (10, 4.5)

- (13.5, 5.5), el cual se utilizó dos veces, para entrar y salir del área rodeada.
- (16, 5)
- (16, 16)

Posteriormente se sintonizaron los parámetros del controlador, asignando un LookaheadDistance de **0.35**, DesiredLinearVelocity de **0.75** y MaxAngularVelocity de **20**; valores utilizados en el conjunto de waypoints anterior.

Se utilizó un valor de maxRange del Lidar de **1.1**, para que el robot pudiera ver antes los obstáculos y tener tiempo para reaccionar

Finalmente, se cambió el tiempo de simulación a **177 segundos**.

Estrategia para reducir el uso de waypoints intermedios

Para reducir el uso de waypoints intermedios, sin que el robot choque o se desvíe de la ruta, consiste en hacer dotar al robot de la máxima autonomía posible, haciendo que sea más flexible al navegar entre puntos.

Esto se podría conseguir aumentando el rango del Lidar, ya que, si el Lidar puede ver los obstáculos con más anticipación como las paredes, el robot se ve obligado a centrarse en los pasillos. Esto ayudaría más que nada a evitar que el robot se desvíe cuando dos puntos están en línea recta pero muy separados, lo cual se corregía con waypoints intermedios. También habría que adaptar el LookaheadDistance, que cambia según la ruya, pero aumentándolo ligeramente, el robot suavizaría su trayectoria evitando girar sobre su eje u oscilar, lo cual se corregía con waypoints intermedios. Por lo mismo también habría que disminuir la velocidad lineal, para que el robot tenga más tiempo de actuar ante obstáculos.

Con esta autonomía, el robot solo necesitaría de waypoints intermedios en puntos ciegos absolutos; por ejemplo, si tiene que hacer una forma de U porque hay una pared, el robot necesitaría de un waypoint intermedio para que el robot pueda encontrar el camino solo.

Otra estrategia de la colocación de waypoints intermedios está relacionada con el diseño de la ruta, haciendo que como se comentaba anteriormente, los puntos ciegos absolutos sean la cantidad mínima posible, para que solo se coloquen waypoints donde sea estrictamente necesario. También, parte de esta estrategia es colocar waypoints intermedios en las “avenidas” principales del mapa, como se vio en el complexMap; haciendo que los waypoints intermedios lo guíen a partes del mapa donde pueda avanzar grandes distancias en línea recta, lo que le permita guiarlo a su destino, y reduciendo el número de waypoints intermedios utilizados.